

第5回 生成AIとAI駆動開発 #1



Technical Debt in ML Applications

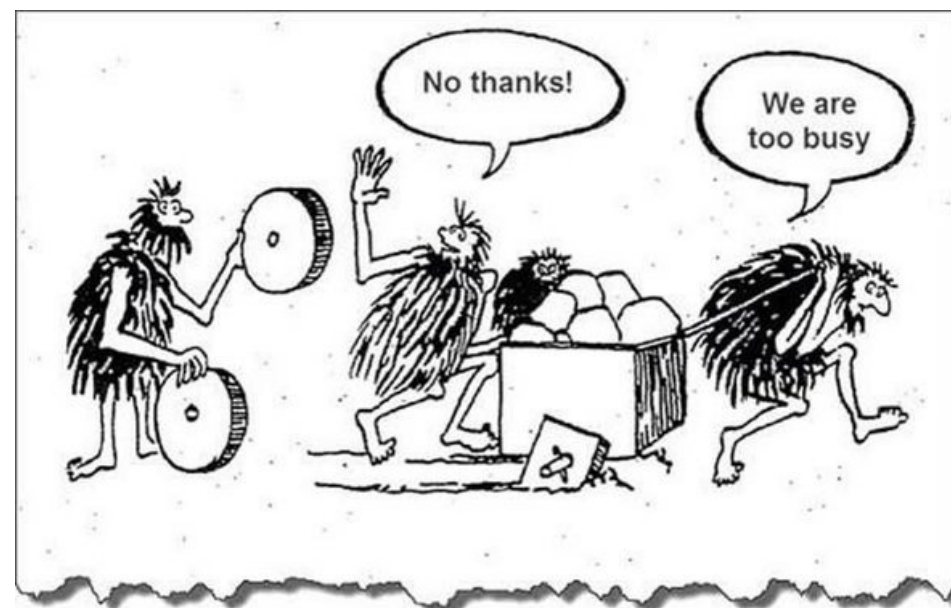


- Traditional testing critically relies on abstraction boundaries of components with specific intended behaviors. The behavior of an ML component depends on external data and is not predictable:

- Change Anything-Changes Everything**

- Data dependencies introduce new constraints on system decomposition and cost more than code dependencies, due to lack of tools, thus breaking good software engineering practices
- System becomes a spaghetti due to glue code that supports ML, unique pipelines for data preparation, conditional code paths for experimentation, etc.
- Maintenance is expensive due to inevitable changes in data and models with time

- Maintenance starts immediately after deployment**



"Machine Learning: The High-Interest Credit Card of Technical Debt", D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary & M. Young, Software Engineering for Machine Learning Workshop, NIPS 2014

ソフトウェア開発における生成AI利用の目的と効果



- AIは、効率、創造性、開発プロセス全体を高めるツールとテクニックを提供します。生成AIは、タスクを自動化し、生産性を高めることで重要な進歩を推進しています。ソフトウェア開発でAIが使用される主な分野は次のとおりです。

主な利用シーン

- コードの生成
- バグの検知と修正
- テストの自動化
- プロジェクト管理
- 資料作成と整合性維持
- リファクタリングと最適化
- セキュリティー強化
- DevOpsとCI/CDパイプライン
- UXデザイン
- アーキテクチャー設計

X

主な利点：

- 生産性向上
- 開発期間短縮
- 品質向上
- 利益率向上（省力化）
- 教育効果（スキル育成）
- 新規性獲得
- 業務知識の永続化

=

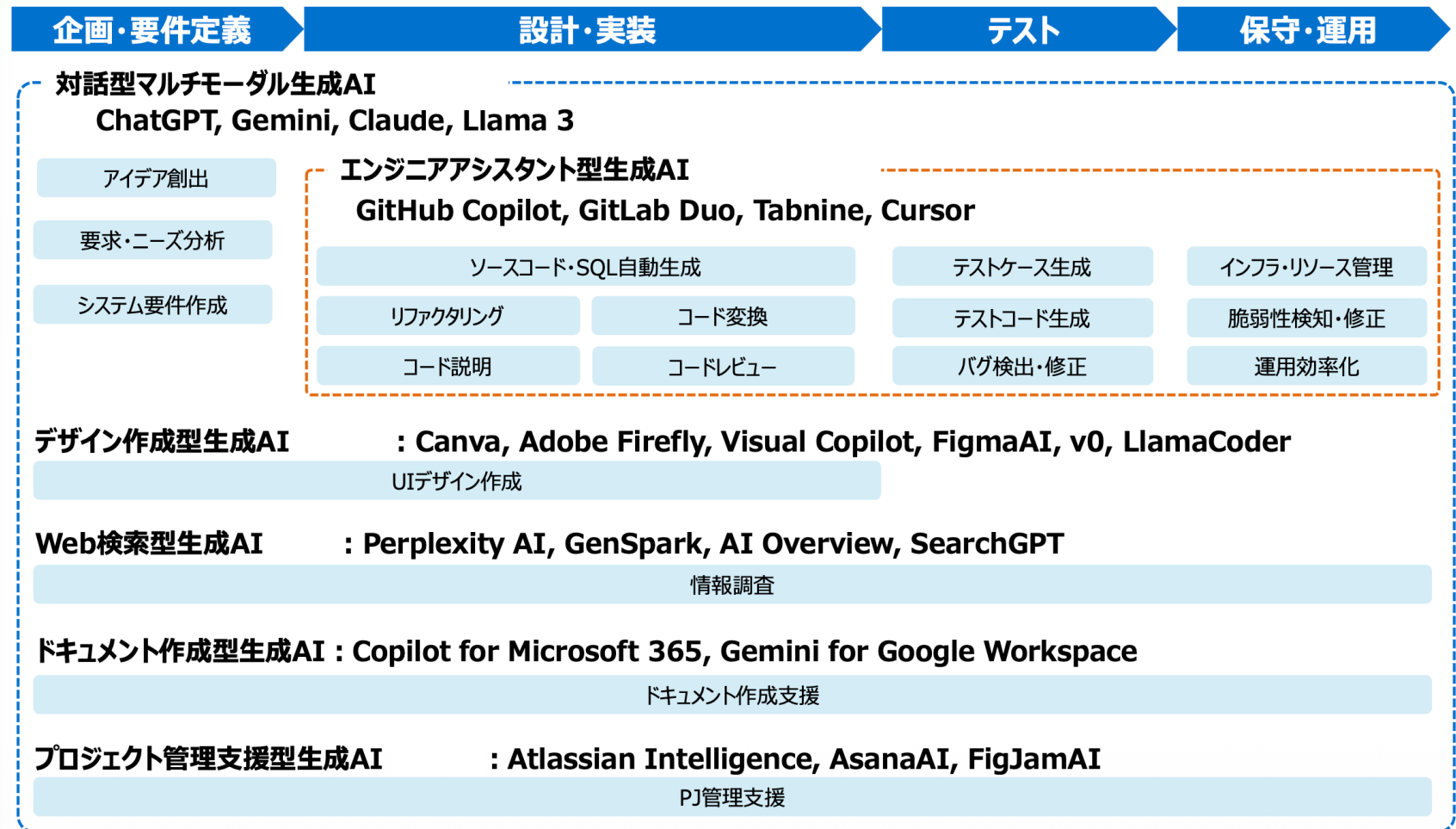
主なゴール：

- **短期的目標**
人間とAIの協調による開発効率化
- **中期的目標**
AI導入が前提となるシステム開発手法の確率
- **長期的目標**
開発内製化のハードルが下がることによるビジネス構造変革

ソフトウェア開発における生成AI利用の目的と効果



- ・ 昨今の対話型の生成AIをまとめると、次のような図になります。



参考
<https://www.jri.co.jp/MediaLibrary/file/advanced/advanced-technology/pdf/15272.pdf>

開発支援AIツールを使い分ける



- AI技術の急速な進化により、多くの技術者がChatGPTをはじめとする対話型AIに注目しています。確かにこうした愛和型のAIツールは強力なツールですが、開発現場ではさまざまなタイプのAIツールが重要な役割を果たしています。効果的な開発のためには各種ツールの特徴を理解し適材適所で活用することが鍵になります。
- 開発支援AIツールは主に3つのタイプとなります。

開発支援ツールの3つのタイプ

自動補完型：

GitHub Copilotに代表される、リアルタイムでコード補完を提供するツール。
コーディング中の生産性を飛躍的に向上させる。

対話型：

ChatGPTに代表されるユーザーとの対話を通じて問題解決をサポートするツール。
複雑な概念の説明やアルゴリズム設計などに威力を発揮。

エージェント型：

GitHub Copilot Workspaceのように、複雑なタスクを自律的に実行するツール。
幅広い汎用タスクに対応可能。

	自律補完型	対話型	エージェント型
実装	集中力を伴うコーディングの支援	ボイラープレートおよびコードの叩き台の作成	複数ファイルに及ぶ実装
改善	トライアンドエラーによる高速改善	コードレビューおよび部分的なリファクタリング	全体的な修正項目の列挙と改善
質問	軽量の指示（コメント）による問いかけ	実装内容の相談や技術的な質問	複合的な対象物へのレビューの実施
変換	コメント作成やサンプルの提示による実装	ドキュメントやコードを別のフォーマットに変換	全体的なプロジェクトの更新（機能や文書の追加）

参考) ソフトウェア開発における生成AI利用の目的と効果



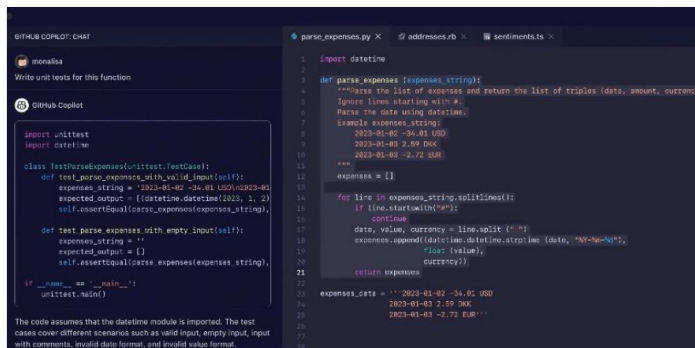
- 代表的なエンジニアアシスタント型生成AIの例をいくつか示します。

GitHub Copilot (GitHub社)

- コーディングを補助する生成AIの代表的サービス。Visual Studio Code^{*1}等のエディタ・IDEと統合されている。
- GitHub上のコードを学習したAIモデルであり、様々な言語のコード生成に対応。**コミットメッセージ生成、プルリクエスト生成、脆弱性修正**など、開発ライフサイクル全体の効率化に寄与。
- ライセンス侵害防止のため、**一定の長さ以上公開コードと一致する場合にはコードの提案を抑える機能**なども提供している。
- 自然言語でコードに関する計画や、ビルド、テスト、デバッグができる「GitHub Copilot Workspace」も提供^{*2}。

^{*1}：以降、「VSCode」と表記 ^{*2}：2024年4月からテクニカルプレビュー版として提供開始

利用例：コーディング中の関数(右)について、単体テスト作成の指示を行い、結果を確認(左)



[出所] GitHub Copilot · Your AI pair programmer · GitHub, <https://github.com/features/copilot> (参照：2024/9/3)



Copyright (c) The Japan Research Institute, Limited

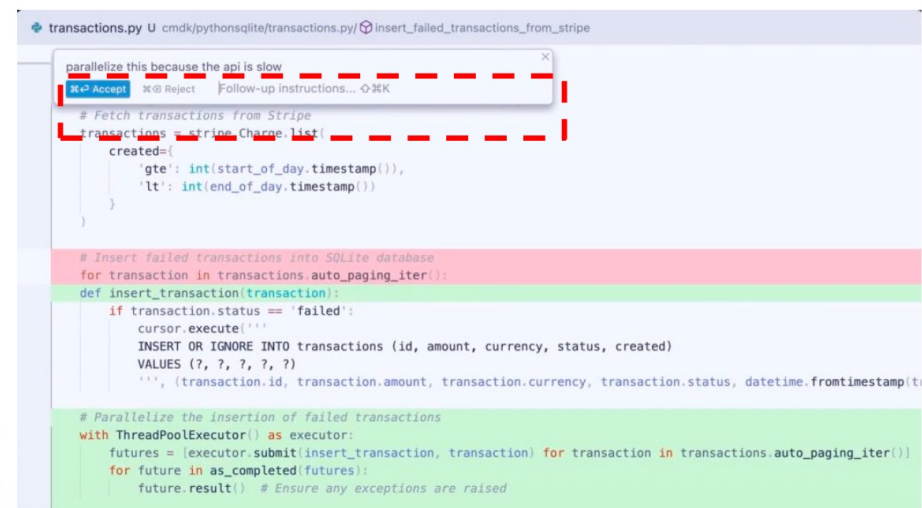
Cursor (Anysphere社)

- VS Codeを複製して開発された**生成AI搭載のコードエディタ**。
- 直感的なUIで使いやすく、継続的に機能改良されている。
- コードやライブラリのドキュメントに関するQ&Aも可能。
- Composer機能により、複数ファイルの編集もでき、ユースケースが広がった^{*3}。

^{*3}：2024年9月時点では、ベータ版として提供

利用例：API処理の並列化

自然言語による入力(下図赤枠)により、生成AIが既存コードの変換を自動で提案する。利用者は「Accept」ボタンにより提案を受け入れる。



[出所] Cursor - Built Software Faster, <https://www.cursor.com/> (参照：2024/9/4)

poration

参考) ソフトウェア開発における生成AI利用の目的と効果

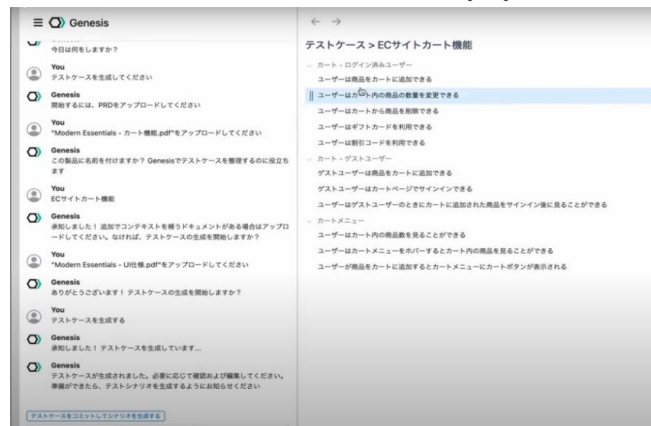


- 代表的なエンジニアアシスタント型生成AIの例をいくつか示します。

Autify (オーティファイ社)

- AIによるノーコードテスト自動化ツールに生成AIを搭載した機能「Autify Genesis」をβ版として公開。
- プロダクトの仕様書をアップロードし、**対話形式でテストケース・テストシナリオを自動生成**。

利用例：仕様書についてLLMと対話し(左)、テストケースを生成(右)



[出所] Autify Genesis
| 生成AIによるテストケース・テストシナリオ自動生成
| Autify (オーティファイ),
<https://autify.jp/products/genesis>
(参照：2024/9/3)

SQLAI.ai (SQLAI.ai社)

- SQLの取り扱いに特化し、**自然言語からのSQL文の自動生成や修正、説明、最適化**が可能。MySQLやPostgreSQL, SQL Serverなど20種以上のDBエンジンに対応。
- 自社のDBスキーマでAIを学習する機能も提供**している。

[出所] Generate SQL Queries in Seconds for Free - SQLAI.ai, <https://www.sqlai.ai/> (参照：2024/9/3)
Copyright (c) The Japan Research Institute, Limited



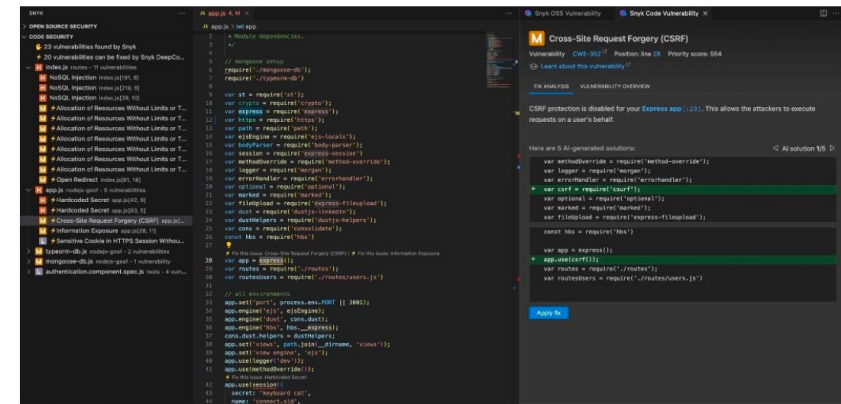
Snyk DeepCodeAI (Snyk社)

- 「Snyk Code」サービスで提供されている、**コードの脆弱性診断と修正案を自動で提示**するサービス。
- AIコーディングツールと組み合わせるユースケースを想定し、生成AIにより出力されたコードの診断も可能。
- 生成AIと複数の機械学習手法の両方を活用して、**ハルシネーションを抑えた高精度なスキャンと修正案生成**が可能。

DeepCodeAIによるCSRF*検知・修正の例

実装したコード(左)からCSRFの脆弱性を検知し、
コード修正案(右)を提示

*クロスサイト・リクエスト・フォージェリ



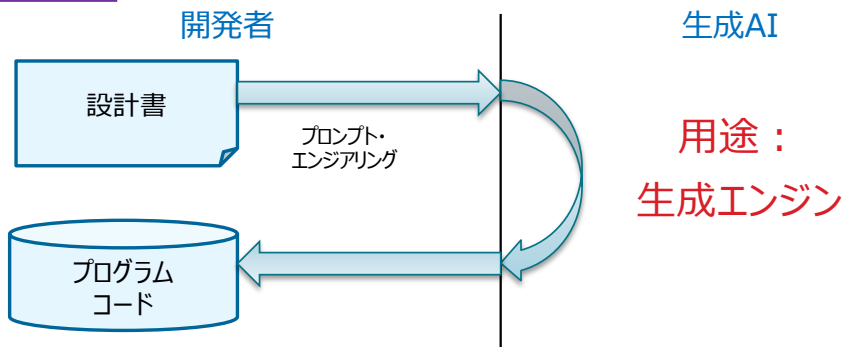
[出所] Fix code vulnerabilities automatically | Snyk User Docs, <https://docs.snyk.io/scan-using-snyk/snyk-code/manage-code-vulnerabilities/fix-code-vulnerabilities-automatically#enable-deepcode-ai-fix> (参照：2024/9/3)

ソフトウェア開発における生成AI利用シーン

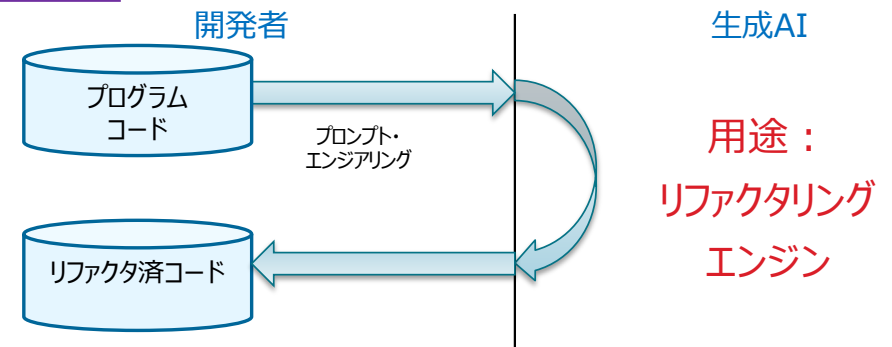


- 開発における生成AIの利用シーンは多岐に渡り、開発活動全般において開発者をサポートすることができます。
- ユーザーとの対話に着目すると、以下のような代表的な開発活動 4 パターンが一般的ですが、これ以外にも、プロジェクト管理や運用における利用、要件の整理など、さまざまなシーンで利用可能だと言われています。

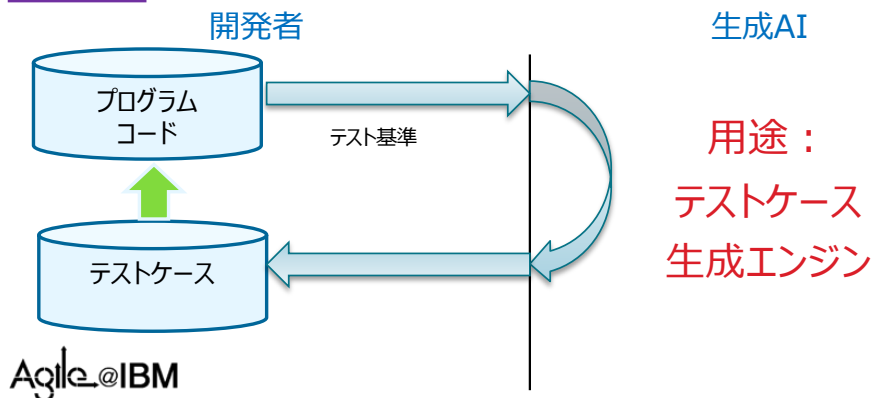
パターン1



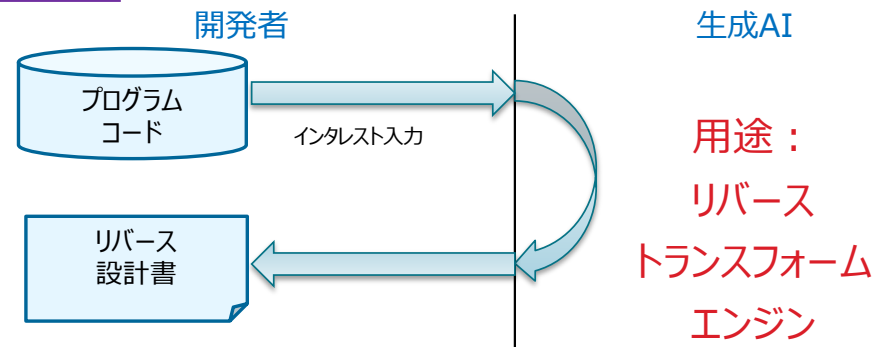
パターン2



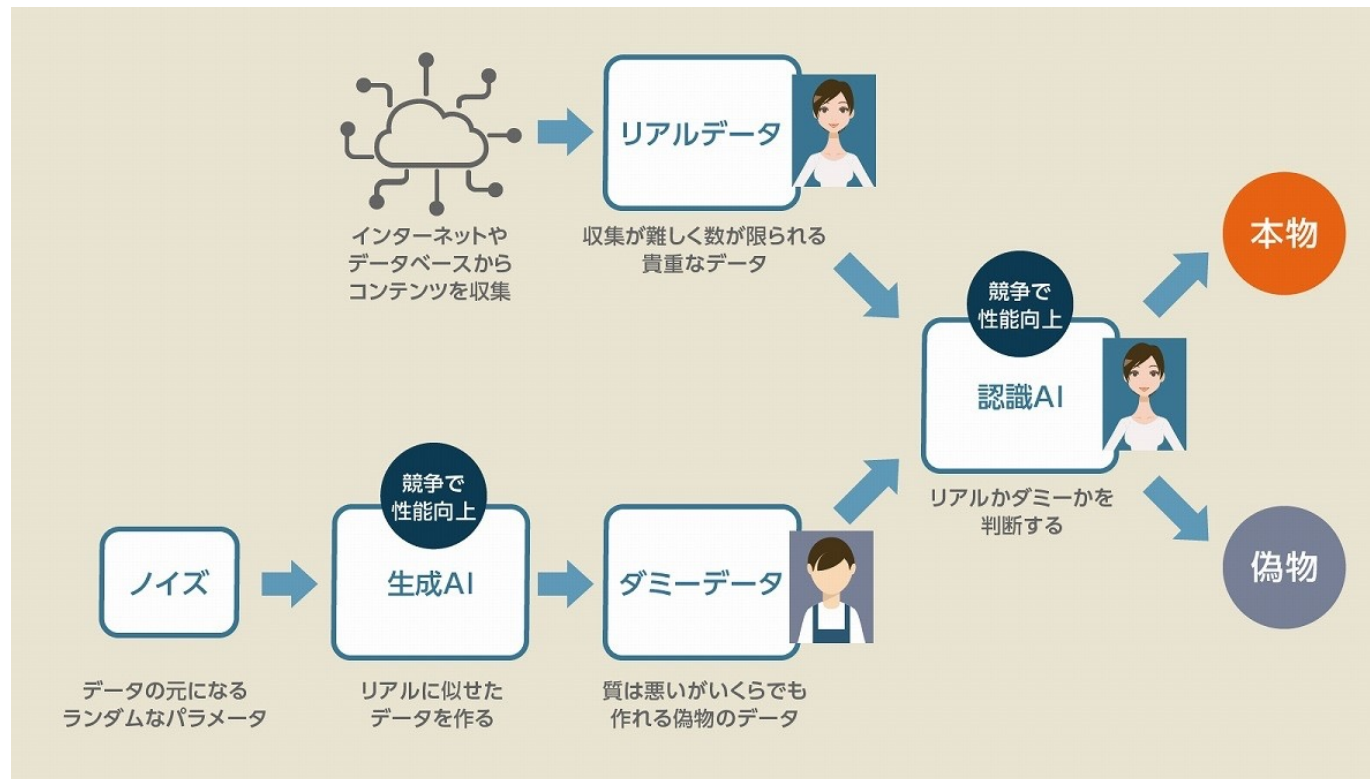
パターン3



パターン4



そもそも生成AI（ジェネレーティブAI）とは？



まず、近年のAIにおける革新は「画像認識AI」の登場から始まりました。これは機械学習とディープラーニングが注目される大きなきっかけとなっています。この時点でのAIのコンテンツ生成能力はまだあらゆる種類がありません。むしろ抽象的な物体を生み出すのが関の山でした。生成AIも人間の子供と同じように、コンテンツを生成する前に、まずは目の前に存在するコンテンツが「どういう存在なのか」を正しく認識できるようになるところから始まったのです。

その認識AIの性能向上の過程で生まれたのが「**GAN（敵対的生成ネットワーク）**」と呼ばれる学習法です。これは認識AIと生成AIを同時に学習させることでAIの精度を上げる学習法で、従来の機械学習では難しかった「学習用データが少ないコンテンツ」の学習を可能にしました。

この手法の特徴は人間が作成・収集した「リアルデータ」と、生成AIが作成した「フェイクデータ」を認識AIに識別させることで認識AIの判定精度を向上させつつ、生成AIのフェイクデータを少しずつリアルに近づけていくことができる点にあります。

そもそも生成AI（ジェネレーティブAI）とは？



■ 生成AIを支える技術：GPT4 / Transformer / LLM（参考 <https://www.sbbit.jp/article/cont1/99349>）

生成AIが進化する決め手となったのが文章生成AIである「GPT-3」の登場です。GPT-3については、簡単にまとめると、新しいディープラーニングの学習法である「Transformer」と呼ばれる手法を使ったGPT（Generative Pre-trained Transformer）シリーズの3代目です。

GPTに使われているTransformerは従来の手法に比べて学習効率においてはるかに優れた性能を発揮し、膨大なデータを一度に使った極めて大規模な自然言語学習を可能にしました。このような学習モデルを「LLM（大規模言語モデル）」と呼び、現代のほとんどのLLMにTransformerが使われています。

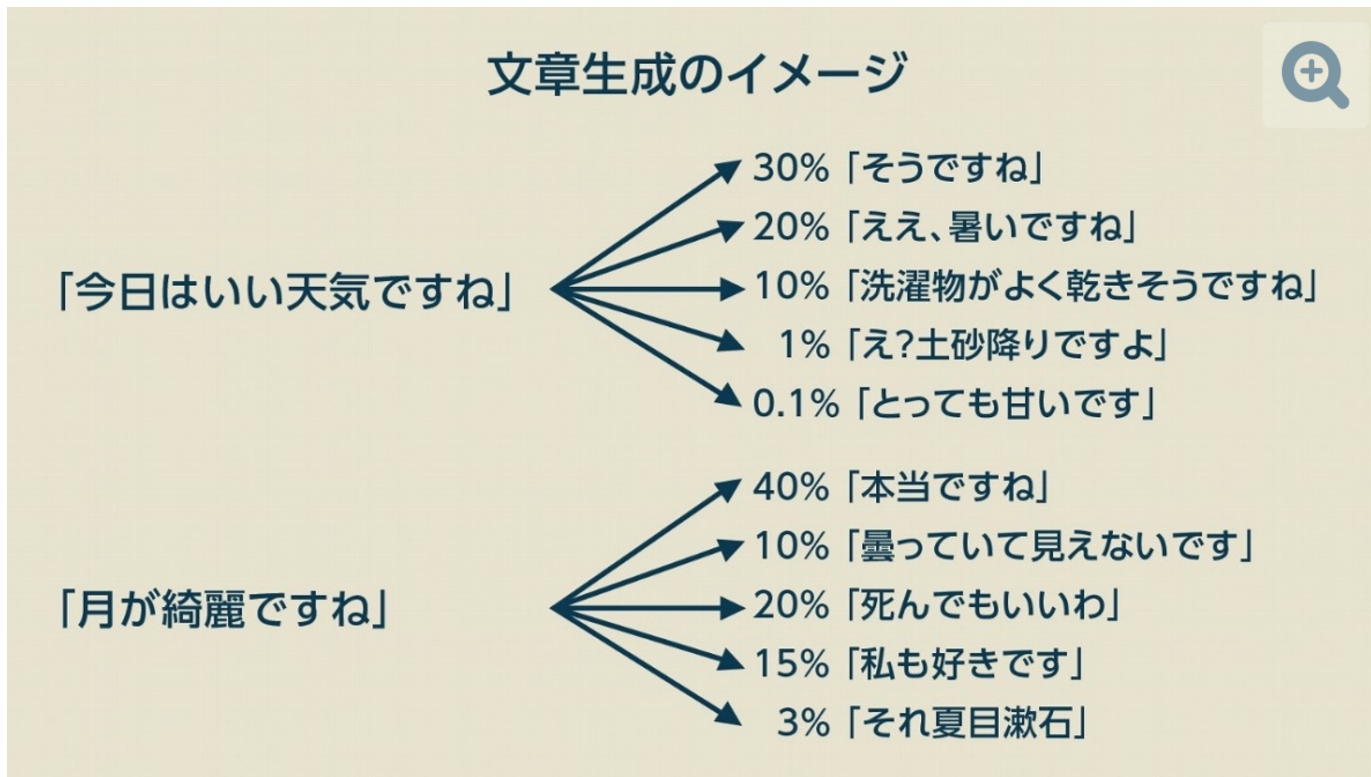
また、こうした自然言語処理の学習モデルでは、言語同士の関係性を表す「パラメータ数」を「扱える複雑さの指標」として用いることがあり、GPT-3は発表当時としては最大規模の1750億のパラメータを扱えるとして話題になりました。基本的にパラメータが増えるほど、使われるニューラルネットワークの規模が巨大になり、計算処理にかかるコストが増大し、学習モデルの調整やチューニングも難しくなります。

LLMの名称	最大パラメータ数	開発企業	発表日	備考
GPT-1	1.2億	OpenAI	2018年6月	GPTの初期モデル
GPT-2	15億	OpenAI	2019年2月	GPT-1の発展型
GPT-3	1750億	OpenAI	2020年6月	人間と同等の文章生成能力
GPT-3.5	非公開	OpenAI	2022年3月	GPT-3の改良版
	(推定1750~3500億)			ChatGPTのベース
GPT-4.0	非公開	OpenAI	2023年3月	BingAI/ChatGPTに導入
	(推定0.3~1兆前後)			マルチモーダル対応
BERT	3.4億	Google	2018年5月	Transformer型LLMの基礎
LaMDA	1370億	Google	2021年5月	Bardのベース
PaLM	5400億	Google	2022年4月	Bardに導入、GPT-3以上の性能
PaLM2	5400億	Google	2022年5月	Bardに導入、マルチモーダル対応
LLaMA	650億	Meta	2023年2月	小規模モデルも提供している
NeMo	5300億	NVIDIA	2022年9月	用途に合わせたモデルを提供
				マルチモーダル対応
GPT-NEOX	36億	rinna	2023年5月	日本語AI「りんな」の企業が開発
Open-CALM	68億	CyberAgent	2023年5月	日本語ベースでは規模が大きい

そもそも生成AI（ジェネレーティブAI）とは？



- コンテンツを生成するしくみ：文章（参考 <https://www.sbb.it.jp/article/cont1/99349>）



一方、文章生成AIが行うタスクの本質は「与えられた文章と関連性の高い文章を生成する」という点にあります。つまり、AIに問題文が与えられた場合、その問題文と関連性の高いものをAIは「問題に対する正しい回答」と考え、「回答文を生成する」といった具合に質問に答えるAIが作られているのです。

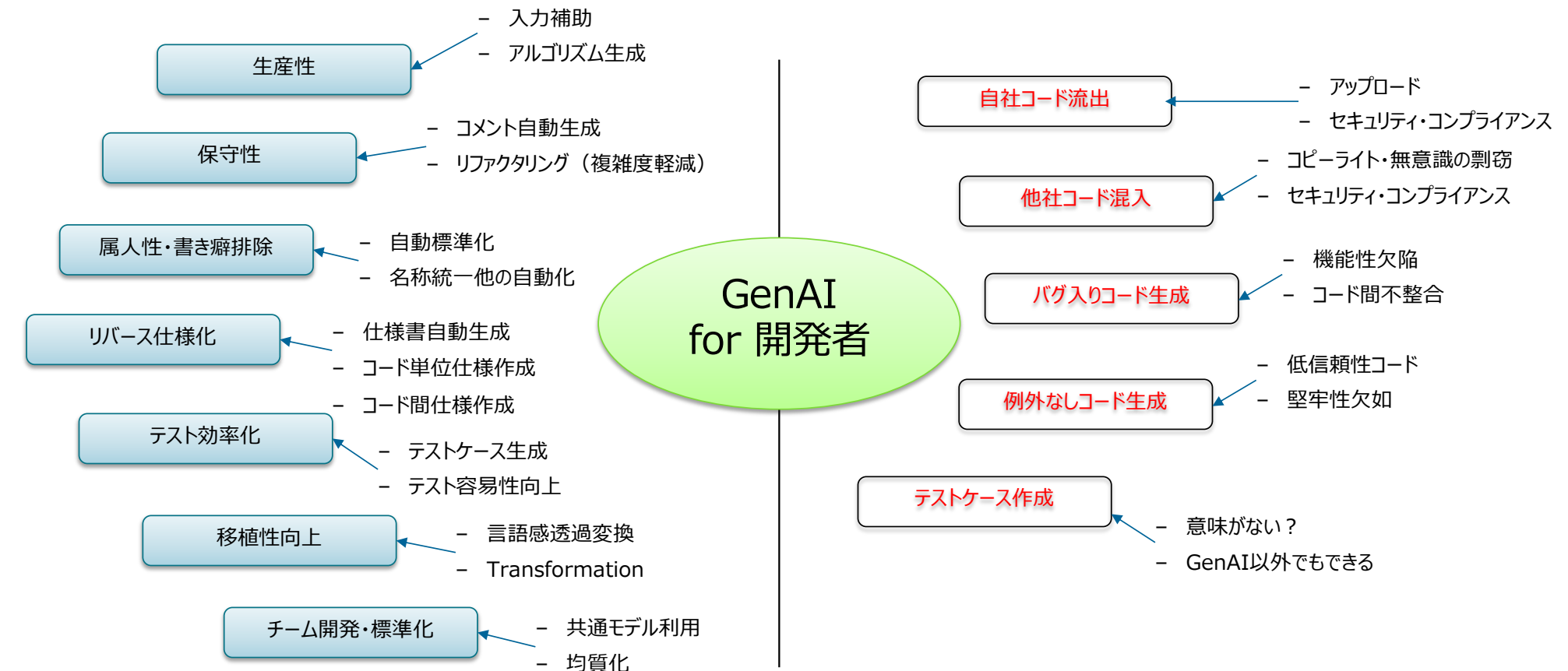
それと同様の仕組みを基に「報告：概要」や「要点：記事」のような対応関係が作れば、求めに応じて関連性の高い「概要文」や「記事」を作ってくれるというわけです。

このため、良い意味でも悪い意味でも「実際の言葉の意味とは異なる文章の流れ」を生成することも可能です。

生成AI利用のリスクと課題（全体像）



- GenAIの開発シーンへの適用は、さまざまなメリット・デメリットがあります。
- 当講義では、メリットとデメリットの両面を公平に解説し、開発者が知るべき「考慮点」について学習します。



生成AI利用のリスクと課題（全体像）



- 前述の「利用シーン」のうちパターン1を例にとり、生成AIのリスクを以下に解説します。
- 開発者がプロンプトで入力した設計情報やプロンプトは、最終的に生成されたプログラムと共に「学習済モデル」として、生成AIのエンジンに取り込まれ、次の利用者がモデルを利用することで「企業内の情報が漏洩」する可能性があります。
- 一方、外部からモデル作成エンジンを経て学習されたコードの断片が、企業内開発者に対して直接結果として返答される可能性があるため「外部のOSSなど「知的所有権違反」となる可能性を有しています。

利用パターン1



- 生成AIによって自動生成したソースコードは、著作物として認められない可能性がある。
- 生成AIによって自動生成したソースコードが、他者のソースコードの著作権を侵害している可能性がある。

生成AI利用のリスク: 典型例 有名なアルゴリズムは学習済である（著作権リスク） IBM

- このコードは生成AIに対する作業指示（プロンプト）として「素数を生成するPythonの”メソッドX”を生成してください」と指示したものです。
- そして、このプログラムに対して「これは何をしているプログラムか解説してください」と解説を依頼する（/explainコマンド）を発行すると右の解説を生成してくれます。

``` **Prompt:** (Github Copilotの例)

ゼロから整数nまでの素数を生成するPythonプログラムを生成してください

### 生成されたコード:

```
def Method_X(n: int) -> List[int]:
 primes = [True] * (n + 1)
 primes[0] = primes[1] = False
 for i in range(2, int(n**0.5) + 1):
 if primes[i]:
 for j in range(i * i, n + 1, i):
 primes[j] = False
 return [i for i in range(2, n + 1) if primes[i]]
```

### 内容の解説（/explain結果）:

このコードは\*\*エラトステネスの篩（Sieve of Eratosthenes）\*\*と呼ばれる古典的なアルゴリズムを実装したものです。指定された数値nまでの全ての素数を効率的に見つけるためのアルゴリズムで、紀元前3世紀にギリシャの数学者エラトステネスによって考案されました。

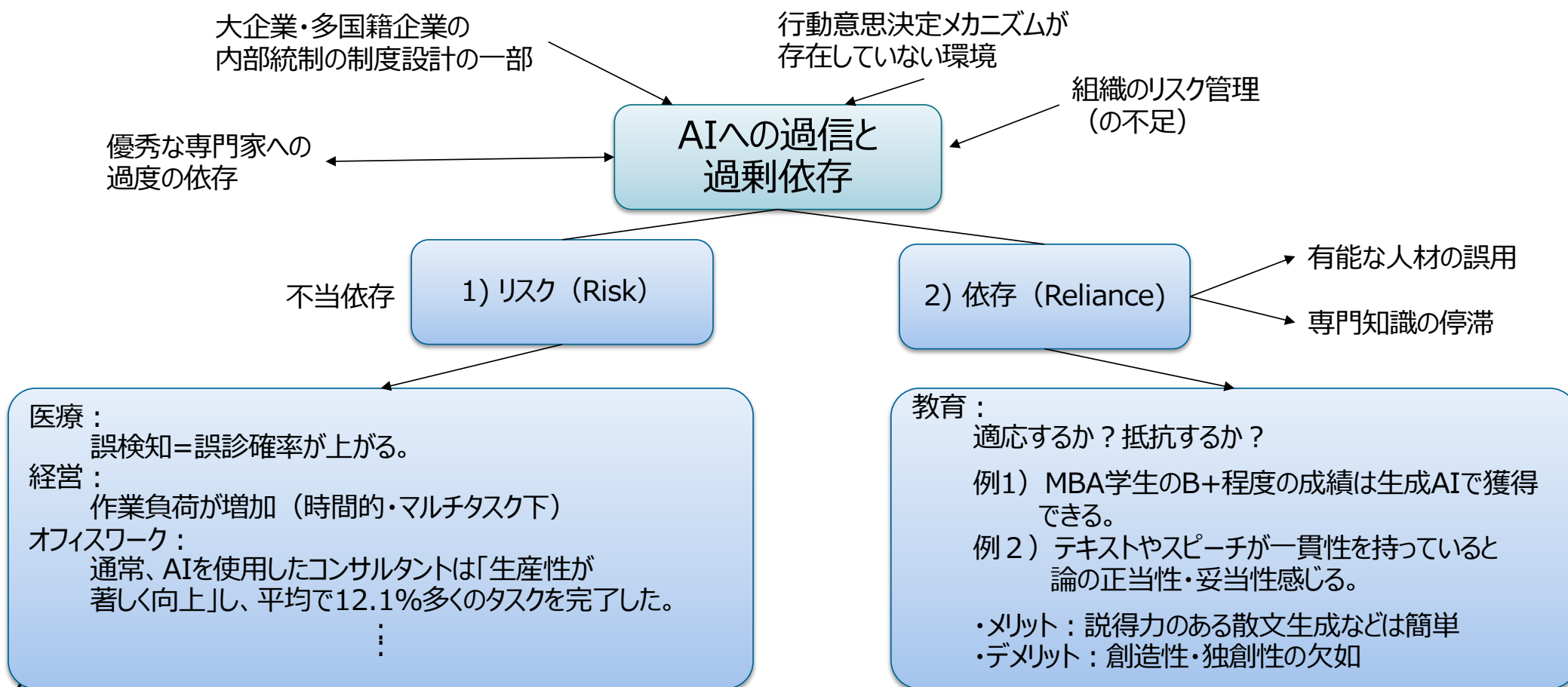
アルゴリズムの動作原理は非常にシンプルです。まず、2からnまでの全ての数を「素数候補」として扱い、小さな素数から順番にその倍数を除外していきます。最終的に残った数が素数となります。コードではprimes配列でこの候補管理を行い、Trueが素数候補、Falseが合成数（素数ではない数）を表しています。

- 過去に「エラトステネスのふるい」という素数列挙アルゴリズムをGithub Copilotが学習済みであることがわかる。
- 生成コードはどこかWeb上や別のユーザー・コードを学習済みだからコードが生成できる。

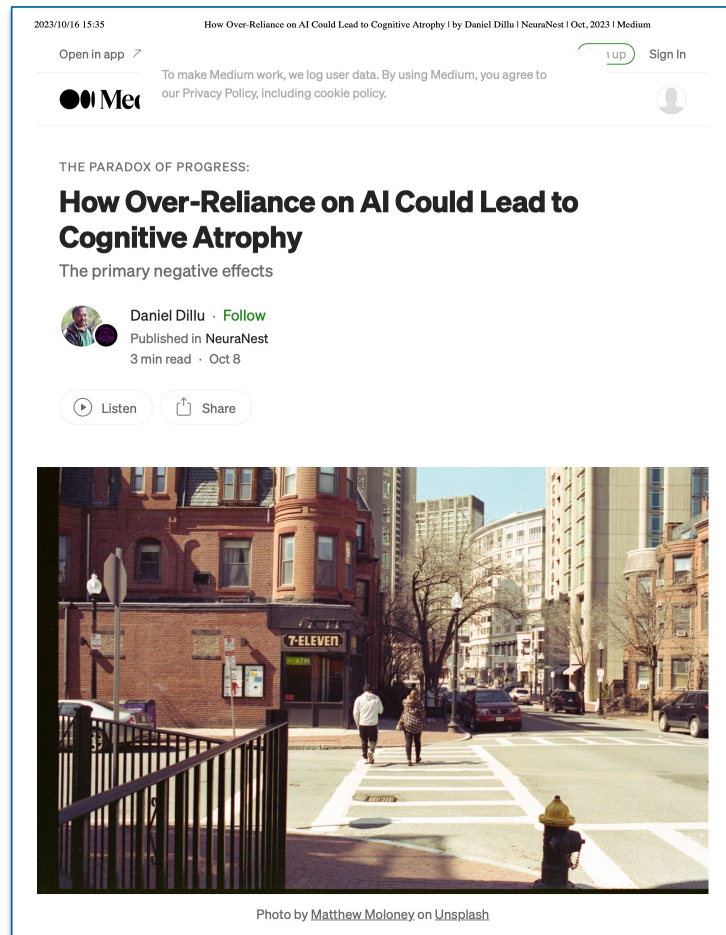
# エンジニアの仕事は無くない: 生成系AIに対する過信 (Over-Reliance)



- 生成系AI=与えられた入力に基づいて「単語」を生成するだけなので入力の「意味」を理解していません。
- このような単純な仕組みかつ他所への著作権リスクなどを考慮すると、過剰な依存や過信は長期的視点でハイリスクです。



# エンジニアの仕事は無くない: 生成系AIに対する過信 (Over-Reliance)



出典 <https://www.itpro.com/technology/artificial-intelligence/over-reliance-on-chatgpt-could-harm-worker-performance>

- 基本スキルの侵食
  - 記憶と想起能力の低下
  - 計算能力の低下 (例: 暗算)
- 批判的思考と問題解決能力の喪失
  - 自動化された意思決定
  - 懐疑心の欠如
  - 均質化
- 探索と学習の必要性を軽減
  - インスタント情報 (情報の範囲が限定的)
  - 過剰専門化 (視野狭窄化の懸念)
- 行動と認知の依存関係
  - フィードバックのループ (の欠如)
  - 注意力の低下 (考察するための精神的労力の減少)
- 曖昧さへの無関心
  - 二項対立的思考
  - 直感の喪失・鈍化

## 例) システム開発におけるLLMの利用

- **\*\*即効性と問題解決能力の低下\*\*** ChatGPTがあなたの成長を妨げる主な方法の一つは、複雑な問題に対して即効性のある解決策を提供することです。これは恩恵のように見えるかもしれませんが、批判的に考え、独自に問題を解決する能力の低下につながります。課題に取り組み、コードをデバッグし、解決策をブレインストーミングするプロセスは、問題解決能力を磨くために不可欠です。ChatGPTだけに頼っていると、この重要なスキルを強化する機会を奪われかねません。
- **\*\*概念の限られた理解\*\*** ChatGPTは、プログラミング言語とフレームワークの幅広い配列のための説明とコードスニペットを提供することができます。しかし、根本的な概念を理解しようとせずに、ChatGPTの解説に頼っていると、使用している技術の深い理解を逃してしまいます。表面的な理解だけでは、短期的には役立つかもしれませんが、予期せぬ問題に遭遇したり、イノベーションが必要になったときに役に立ちません。
- **\*\*外部支援への依存\*\*** ChatGPTに解決策を求めるのが常習的な開発者は、依存のサイクルに陥ってしまうかもしれません。これは、自分自身の能力に対する自信の欠如につながり、独立した開発者としての成長を阻害する可能性があります。時間が経つにつれて、外部からの支援を求める習慣は、自分の仕事にオーナーシップを持ち、十分な情報に基づいた意思決定をする能力を妨げるかもしれません。
- **\*\*創造性と革新性を阻害する\*\*** ソフトウェア開発は問題を解決することだけではありません。ChatGPTの提案に頼ることは、創造的思考を阻害する可能性があります。既成の答えを求め続けることで、斬新なアプローチや新鮮なアイデアを開発するチャンスを逃してしまうかもしれません。



### Chat GPT

## The Perils of Over-reliance on ChatGPT in Your Development Journey.

ABHISHEK SINGH CHAUHAN  
Teaching Assistant at Coding Ninjas  
公開日: 2023年8月26日

+ フォロー

In today's fast-paced digital age, the field of software development is evolving at an unprecedented rate. As developers, we are fortunate to have access to a wealth of resources and tools that can aid us in our journey to create innovative and efficient solutions. One such tool that has gained significant popularity is ChatGPT, an AI-

<https://www.linkedin.com/pulse/perils-over-reliance-chatgpt-your-development-journey-singh-chauhan>

1/6



## 例) AI (特に生成系AI) への依存リスク



最近、ニューヨークの弁護士が、存在しない6つの判例を裁判の準備書面で引用したという事件を目にした。AIが生成したコンテンツを使用する際に、私たちはどのように注意しなければならないかについて考えさせられました。AIは完璧ではなく、間違いを犯す可能性があることを忘れてはならない。

AIの役割：代用品ではなく道具。私は、AIを代用品としてではなく、私たちの能力を補強する道具として捉えるべきだと考えている。

私たちが責任を持ってAIを利用する方法について、次のような考えがあります。

- **AIの透明性**： AIが生成したコンテンツの存在や注入、またAIモデルを使ってどのように意思決定が行われているかをユーザーが理解できるような、透明性のあるAIシステムの開発を奨励する。これにより、コンテンツや意思決定がAIシステムの影響を受けていることをユーザーに確実に認識させることができる。
- **AIが生成したコンテンツのダブルチェック**： 法的手続きやビジネス上の意思決定などの重要な場面で使用する前に、AIシステムが提供する事実や情報が正確であることを確認する。
- **人間による監視を維持する**： AIシステムはエラーを起こす可能性があるため、意思決定プロセスには人間の関与が不可欠である。人間が最終的な決定に責任を持ち、AIが生成したコンテンツが望ましい結果に沿うようにする。
- **AIの限界について学ぶ**： AIシステムの限界と潜在的なバイアスを理解することは、AIをいつ、どのように業務に活用するかについて、十分な情報を得た上で意思決定するのに役立つ。
- **コラボレーションを重視する**： AIと人間が協力して最良の結果を達成できるような協働環境を育成する。結局のところ、AIは私たちの生産性と効率を向上させる可能性を秘めているが、私たちは責任を持ってAIを使用し、AIの支援と人間の関与のバランスを取る必要がある。

Agile@IBM



### The Risks of AI Overreliance



Ashish Bhatia

Product Manager @ Microsoft

公開日: 2023年6月5日

+ フォロー

I came across this recent incident of New York lawyer cited six nonexistent cases in a court brief, which were actually generated by an ChatGPT. This got me thinking about how we need to be cautious while using AI-generated content.

AI will be and is being a game-changer in many aspects of our lives, from drafting

<https://www.linkedin.com/pulse/risks-ai-overreliance-ashish-bhatia>

© 2017 International Business Machines Corporation

## 生成AI利用のリスク: 典型例#2 ハルシネーション対策



- この例は「大安」の日付と「一粒万倍日」の日付を算出するプログラムを生成する例です。

**Prompt:** (ChatGPT3の例)  
「大安」の日付と「一粒万倍日」の日付を算出Pythonプログラムを生成してください

### 生成されたコード:

```
def calculate_daiansoku(year):
 """大安祈祷日を計算する関数"""
 # 大安祈祷日は毎年7月15日
 daiansoku_date = datetime.date(year, 7, 15)
 return daiansoku_date

def calculate_ichiryubambai(year):
 """一粒万倍日を計算する関数"""
 # 一粒万倍日は毎年10月17日
 ichirubambai_date = datetime.date(year, 10, 17)
 return ichirubambai_date
```

- 「大安祈祷日」という**存在しない概念**を作り出しています。大安は特定の日付ではなく六曜の一つで周期的に現れます
- 一粒万倍日も同様に**固定の日付**ではありません。

### 内容の解説 (/explain結果):

このPythonコードは**日本の暦（こよみ）**における縁起の良い日を計算する関数を実装しています。具体的には、2つの特別な吉日を計算する機能を提供しています。

#### 各関数の詳細解説

##### 1. calculate\_daiansoku(year) 関数

**目的:** 「大安祈祷日」の日付を計算

**仕様:** 毎年7月15日固定

**戻り値:** 指定年の7月15日のdatetime.dateオブジェクト

##### 2. calculate\_ichiryubambai(year) 関数

**目的:** 「一粒万倍日」の日付を計算

**仕様:** 毎年10月17日固定

**戻り値:** 指定年の10月17日のdatetime.dateオブジェクト

#### 日本の暦における意味

**大安（たいあん）** は六曜の一つで「大いに安し」という意味があり、何事を行うにも良い日とされています。結婚式や開業、引っ越しなどの重要な行事に選ばれることが多い縁起の良い日です。

**一粒万倍日（いちりゅうまんばいび）** は「一粒の粳（もみ）が万倍にも実る稲穂になる」という意味から、何事始めるにも良い日とされています。特に金銭に関することや新しい事業を始める際に重視される吉日です。

開発者が今後AIと共創する環境になった場合、コードを書く技術よりも読む技術の方が強く求められるようになります。

## 技術・品質面のリスク: ハルシネーション



- AI のハルシネーションとは、AI モデルが生成する不正確な結果や誤解を招く結果のことです。
- これらのエラーは、不十分なトレーニング データ、モデルによる誤った仮定、モデルのトレーニングに使用されるデータのバイアスなど、さまざまな要因によって発生する可能性があります。

| 種類          | 説明                                                                                                                                                       |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 事実ハルシネーション  | AIが <b>実際には存在しない情報を事実として提示する現象</b> です。例えば、歴史的な人物やイベントについての質問に対して、実在しない出来事や架空の人物を作り出すことがあります。<br>これは、モデルが学習データから誤った関連付けを行い、信憑性のない情報を生成することが原因です。          |
| 文脈ハルシネーション  | 質問の意図や前後の流れを誤って解釈し、 <b>文脈にそぐわない回答を返す現象</b> です。たとえば「AIの医療利用」に関する質問に対して、「画像生成AIの活用事例」など別の分野の回答をしてしまう場合があります。これは、ユーザーの意図を正しく把握できず、関連性の低い知識を結びつけてしまうことが原因です。 |
| 構造ハルシネーション  | 正しい質問に対しても、 <b>AIが誤った構造やフォーマットで回答を生成する現象</b> です。例えば、リスト形式で答えるべき質問に対して、AIが段落形式で応答したり、期待される数値やデータをテキストの形で提供してしまう場合です。<br>これは、AIが適切な回答形式を誤認した場合に発生します。      |
| 創造的ハルシネーション | <b>完全に架空の情報や新しい概念をAIが生成する現象</b> です。たとえば、科学的な質問に対して、現実には存在しない理論や用語を作り出すことがあります。<br>これは、AIが「創造的」な思考を行う際に事実を無視して、独自の回答を生成することにより生じます。                       |

### ハルシネーションが起こる原因

#### ハルシネーションの 5 大発生原因

1. 学習データの誤り
2. 文脈を重視した回答
3. 情報が古い
4. 情報の推測
5. 評価方法の問題

### ハルシネーションの完全な対策は困難

- ・ハルシネーションは完全に防止することは現状の技術では不可能。
- ・評価基準そのものが「不確実でも回答することを促す設計」になっている。
- ・企業や個人が生成AIを利用する際は「ハルシネーションは一定確率で必ず起きる」という前提を持ち、リスクを低減する仕組みを導入することが必須。

### ＜対策＞

- ・ **出力内容のファクトチェックを徹底する**
- ・ **AIリテラシーを社内外で浸透させる**
- ・ **ガイドラインやレビュー体制を整備する**

## 技術・品質面のリスク: ハルシネーションが発生しやすい質問とその対策



- 生成AIは万能ではなく、質問の仕方によって誤情報を出力する確率が高まります。特に以下のようなケースでは、ハルシネーションが発生しやすいので注意が必要です。

| 質問事項              | 原因                                     | 対策                                   |
|-------------------|----------------------------------------|--------------------------------------|
| プロンプトに嘘の情報が含まれる質問 | AIは入力を事実確認せずに「前提」として扱うため。              | 「この人物や理論は実在しますか？」と存在確認を先に促すと誤りを減らせる。 |
| 架空のことにに関する質問      | 事実が存在しない場合でも「整合性のある物語」を優先する設計になっているため。 | 「存在しない場合は存在しないと答えて」とプロンプトで明示する。      |
| 歴史や時事問題に関する質問     | 知識のカットオフ（学習範囲の限界） + 曖昧な記憶の補完がある。       | 最新の情報は公式サイトや一次ソースで必ず確認。              |

### ハルシネーション対策

#### ハルシネーションを防ぐための質問のコツ

- 存在確認を最初に指示する（「存在するかどうか教えて」）
- 出典やURLと一緒に提示するよう要求する
- 年代や数値は「表形式で」「根拠付きで」と指定する
- 「わからない場合はその旨を答えて」と条件を加える

これらを組み合わせることで、AIが無理に答えを作らず、不確実性を表明するよう誘導できます。



## 技術・品質面のリスク: ハルシネーションを予防する方法 : ダブルチェック (再確認) 機能 IBM

- Gemini (旧Google Bard) (<https://weel.co.jp/media/googlebard-usage>)が2023年9月のアップデートで、「**ダブルチェック**」という機能を実装しました。
- これは、生成した内容をGoogle検索し、生成した文章のどの部分が正しい情報で、どの部分がAIによるハルシネーションなのかを識別できる機能です。これにより、AIによる生成内容をAI自身が検知できるようになりました。

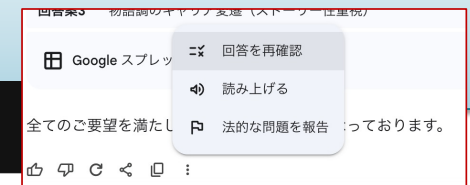
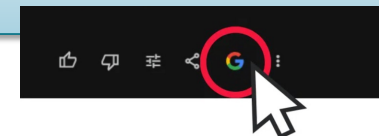
### ``プロンプト

大谷翔平について、経歴や成績を教えてください  
回答案を3種類作成してください



### ``プロンプト

ダブルチェック



### 上記プロンプトからGeminiから回答が得られる

#### 回答案1 : 簡潔にまとめたプロフィールと最新成績

大谷翔平選手は、野球の常識を覆した「二刀流」のスーパースターです。高校卒業後、日本のプロ野球（NPB）を経てMLBへ移籍し....

#### 回答案2 : 詳細なキャリアマイルストーンと通算成績を強調した回答

大谷翔平選手は、類まれな才能と実績により、現代野球で最も注目される選手です。彼のキャリアは「二刀流の確立」「歴史的なMVPシーズン」「世界一への貢献」という3つのフェーズで語られます...

#### 回答案3 : 物語調でキャリアの変遷を語る回答

大谷翔平選手の野球人生は、まさに「異次元」という言葉がふさわしいものです。彼は岩手県の花巻東高校時代から、投手と打者の両方をこなす**二刀流**として注目を集めました。2013年に...

Agile@IBM

| 回答案  | 特徴                          | 評価 |
|------|-----------------------------|----|
| 回答案1 | 簡潔なプロフィールと最新成績（表形式）         | OK |
| 回答案2 | 詳細なキャリアマイルストーンと通算成績（受賞歴を強調） | OK |
| 回答案3 | 物語調のキャリア変遷（ストーリー性重視）        | OK |

## 今回の例題) 複雑なIF条件文の検出と整理



1950年代の古典アルゴリズムによる「複雑なIF文の自動検出と等価書き換えによる簡略化」

課題:

- 複雑なIF条件文等は、経年の条件式追加・修正によって正しさが証明できない。
- 中身を理解できる人が少なくなっている。
- 潜在的に冗長性を含む確率が上がる。

アプローチ:

- 複雑な論理式 (AND/ORやNOTを用いた長いIF文条件式) を検出する。

解法:

ブール演算を用いた等価書き換えを行い「条件式を簡略化」できないか?  
(Quine-McCluskey法、Petirck法…)

複雑なIF条件式を簡略化できる = 等価書き換えが可能

### ■ AND-OR形論理式

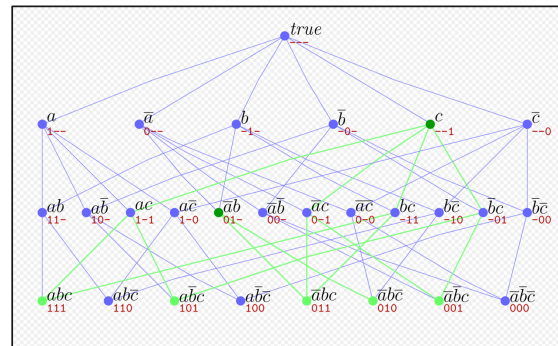
- AND項を+ (OR記号) で結合

(例)

$$f = \bar{x}_1 x_2 x_3 + x_1 \bar{x}_2 x_3 + x_1 x_2 \bar{x}_3$$

- 簡単化のポイント: 圧縮操作

$$xP + \bar{x}P = (x + \bar{x})P = 1 \cdot P = P$$



Willard Van Orman Quine

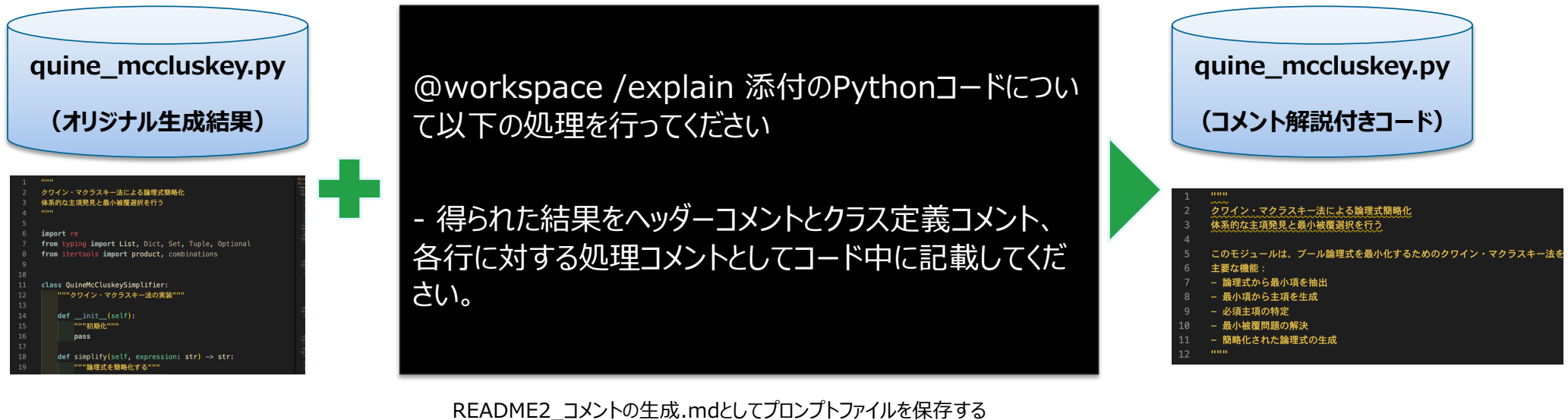
Born: 25 June 1908 in Akron, Ohio, USA  
Died: 25 Dec 2000 in Boston, Massachusetts, USA

[https://en.wikipedia.org/wiki/Quine%E2%80%93McCluskey\\_algorithm](https://en.wikipedia.org/wiki/Quine%E2%80%93McCluskey_algorithm)  
© 2017 International Business Machines Corporation

## コメント生成（パターン1）：ソースにコメント・解説を付記する



- 生成したコードについて、コメントを付記します。それぞれの生成AIツールによって解説の仕方やコメントは異なりますが、以下にGithub Copilotの「/explain」を使って処理内容を解説させる方法を解説します。
- 対象は生成されたコードのうち、「**quine\_mccluskey.py - クワイン・マクラスキー法**」を対象にします。



- 一般的に、生成AIを用いてソースコードを生成した場合、処理内部が適度に構造化されます。しかしプログラミング言語に関係なく、内部構造やどのような仕組みで処理を行っているか「開発者にとってわかりにくい」コードが生成されます。
- 処理内容の解説や仕様の中身をコメントとしてソースコードに挿入しておくことは、保守性向上の観点で極めて重要です。
- コードに対しての処理記述はもとより、再生成や保守運用上の観点でプロンプトファイルも必ず保存しておいてください。**

# リファクタリング（パターン2）：ソースコードのリファクタリング



- コードの改善点を列挙して、ソースコードのリファクタリングを行います。コツは複数の改善案を提示させて、それぞれどのようなメリットデメリットがあるかを生成AIに解説させることです。

@Workspace

当該ソースコードをリファクタリングする手法を3種類のリファクタリングを提案してください:

## 1. ソースコード分割を行い構造を単純化すること最優先とする提案

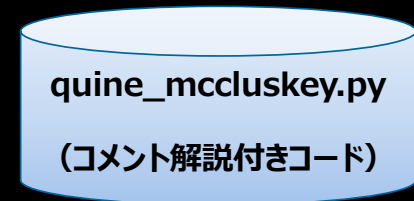
- 複雑で冗長な計算ロジックを別ファイルにすることで循環的複雑度を低減する
- 読みにくい箇所を別ファイルにすることで認知的複雑度を低減する

## 2. 保守性( = 変更容易性、理解容易性、および可読性)向上を最優先とする提案

- 変数命名規則や内部構造の再構造化等を通じて保守性を向上する
- 可変箇所と不変箇所を分離し、拡張性を向上する

## 3. 試験性向上を最優先とする提案

- 複雑で冗長な計算ロジックを整理することで循環的複雑度を低減する
- データフローの単純化により複雑度を低減する
- 複合条件式を分解・単純化することで複雑度を低減する



### \*\*追加要件:\*\*

- 上記3種類の提案は、それぞれ別のファイルとして実装サンプルを提示してください
- 実装結果を「コード行数」「循環的複雑度」「認知的複雑度」「メソッド数」を参考に比較表を作成してください
- 各種データの計測方法は、QI\_Python\_LOC.py, QI\_Python\_CyclomaticComplexity.py, QI\_Python\_CognitiveComplexity.py, QI\_Python\_Method.pyのロジックで計測してください（必要に応じて対象コード修正してください）

結果の比較表はCSV形式で出力して保存してください



## フォワード生成（パターン1）：コード生成（機能拡張）



- 生成の元になるプロンプト文は、直接コマンドとして生成AIにタイプするのではなく、プロンプトファイルとして保存します。保存したら（この例ではGithub Copilot）を利用して

添付の2つのPythonコードに対してプログラムの基礎統計数値を測定します。

- それぞれのコードに対して以下の基礎数値を計測するコードを作成します。
  - LOC (Lines Of Code)とNCSS (Non-Commentary Source Statement)
  - # of Null LinesとComment Rate
  - Number of Operand とNumber of Operater
  - Number of innerClass とNumber of Method
  - Cyclomatic Complexity (Class Total)とCognitive Complexity (Class Total)

それぞれの処理は計測値毎に別々のPythonコードとして生成され、全体をQI\_Main.pyから呼び出される形で実装してください。  
また、それぞれのファイル名は"QI\_Python\_LOC.py"などのファイル命名基準で保存してください。

- 2つのコードに対して計測した1.の結果値を以下のCSV形式にまとめ「QI\_Base\_Statistics.csv」として保存します。

```
Metrics_id, "metrics_name", "file1(Original)", "file2(Original)"
1, "LOC", 100, 200
2, "NCSS", 50, 70
:
```

- 可能な限り標準ライブラリによって実装してください。（3rdパーティー製モジュールへの依存性を極力へらしてください）

## リファクタリング（パターン2）：ソースコードのリファクタリング



- 各種の改善案が提示され、それぞれにメリットデメリットが列挙されました。

| 提案名    | LOC | 循環的複雑度 | 認知的複雑度 | メソッド数 | 総合評価  |
|--------|-----|--------|--------|-------|-------|
| オリジナル  | 451 | 79     | 206    | 10    | -     |
| 構造単純化版 | 104 | 11     | 15     | 3     | ★★★★★ |
| 保守性向上版 | 512 | 97     | 170    | 22    | ★★★★☆ |
| 試験性向上版 | 404 | 107    | 80     | 46    | ★★★☆☆ |

### 構造単純化版（推奨度：★★★★★）

- 優秀な複雑度削減：全メトリクスで大幅改善
- 適用シーン：大規模システムでの複雑度管理
- メリット：保守性・可読性・拡張性の向上
- デメリット：複数ファイル管理の必要性

### 保守性向上版（推奨度：★★★★☆）

- 高い保守性：明確な命名規則と構造設計
- 適用シーン：長期保守が重要なシステム
- メリット：設定分離・状態管理・拡張性
- デメリット：コード量増加

### 試験性向上版（推奨度：★★★☆☆）

- 高いテストビリティ：小さなメソッドで単体テスト容易
- 適用シーン：高品質が要求される製品
- メリット：認知的複雑度大幅改善・テスト容易性
- デメリット：メソッド数大幅増加

### ## 結論

最適な選択は用途による：

1. 複雑度削減が最優先 → 構造単純化版
2. 長期保守性重視 → 保守性向上版
3. テスト品質重視 → 試験性向上版



- これらの提案は排他的に実現する必要はない。組み合わせてコード品質の向上を目指す。
- 開発者の保有するスキル・経験値が3種類のリファクタリングを「どの順序で実施するか？」によってもことなる結果が生成される。

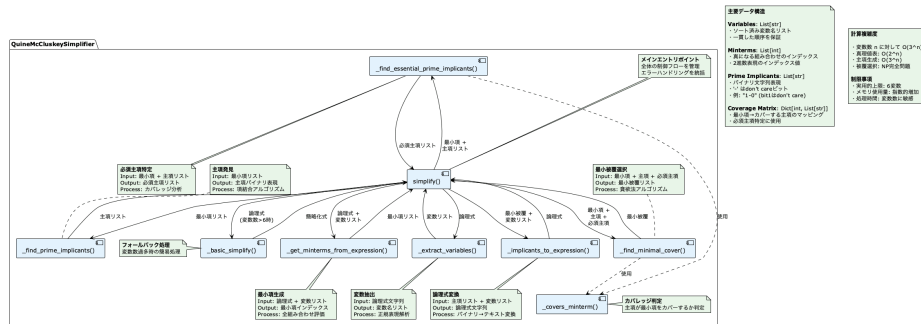
# リファクタリング（パターン2）：ソースコードのリファクタリング



- リファクタリング提案ごとのアーキテクチャ図の比較：各種リファクタリング方針によって全く異なる構造へ。

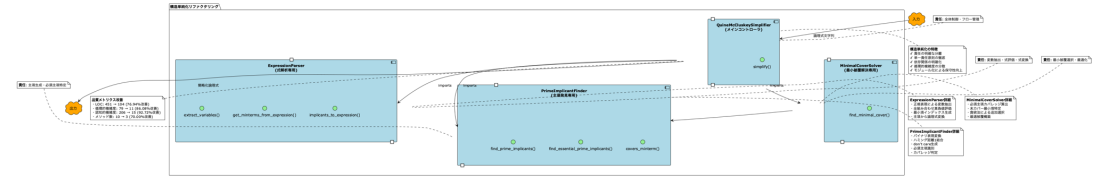
## オリジナル

クワイン・マクスキー法 アーキテクチャ・データフロー図



## 構造単純化版

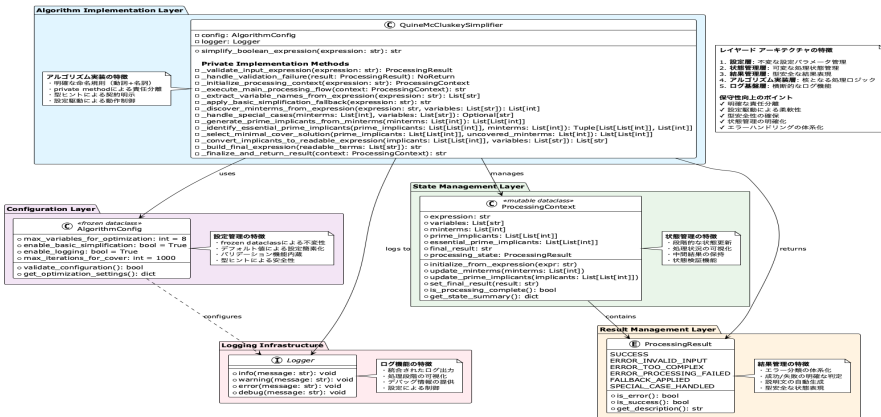
クワイン・マクスキー法 構造単純化版アーキテクチャ図



- モジュール分割して別ファイルへ外出し
- 一つ一つが小さいコードへ分割

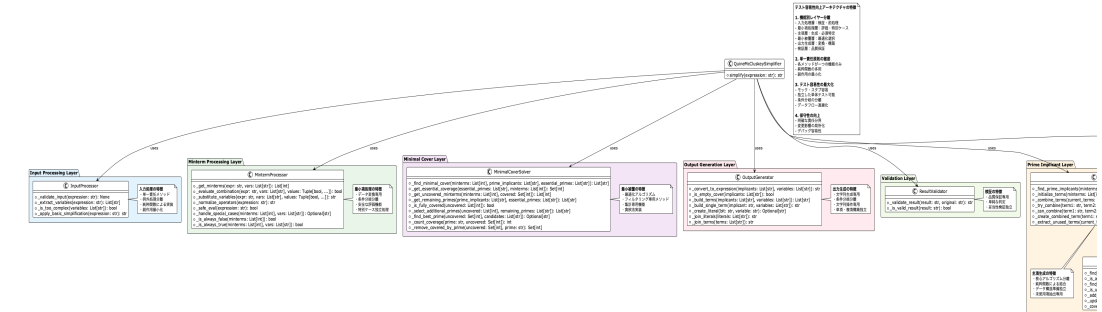
## 保守性向上版

クワイン・マクスキー法 保守性向上版アーキテクチャ



## 試験性向上版

クワイン・マクスキー法 テスト容易性向上版アーキテクチャ



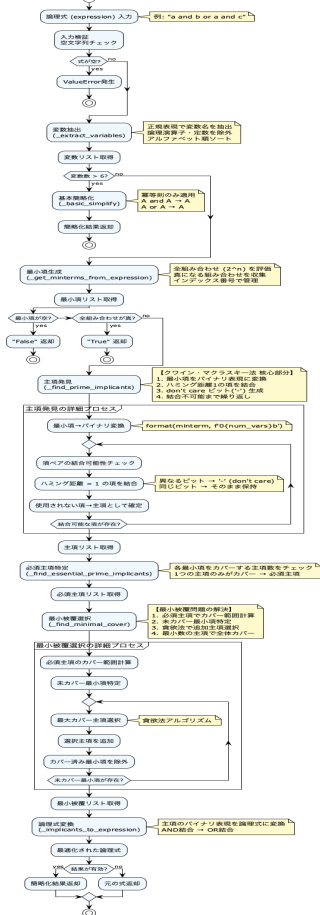
# リファクタリング（パターン2）：ソースコードのリファクタリング



- リファクタリング提案ごとのフローチャートの比較：各種リファクタリング方針によってことなる処理フローへ。

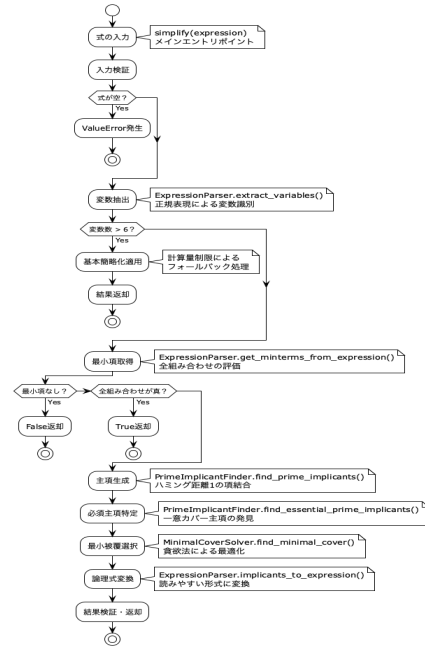
## オリジナル

クワイン・マクラスキー法 データフロー図



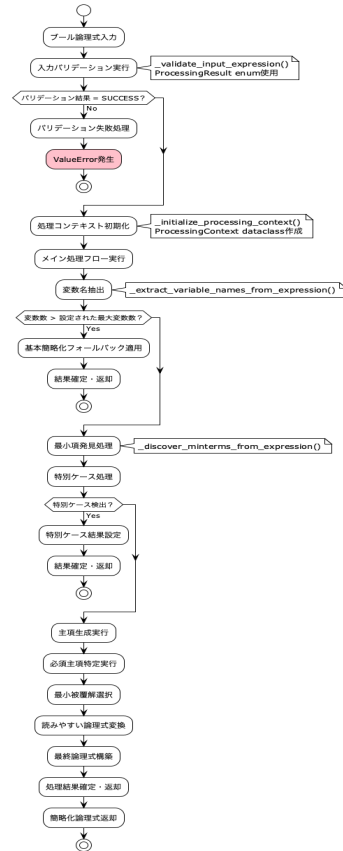
## 構造単純化版

クワイン・マクラスキー法 構造単純化版フローチャート



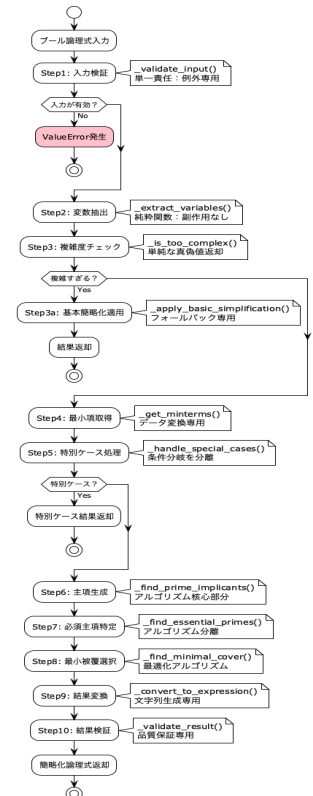
## 保守性向上版

クワイン・マクラスキー法 保守性向上版フローチャート



## 試験性向上版

クワイン・マクラスキー法 テスト容易性向上版フローチャート



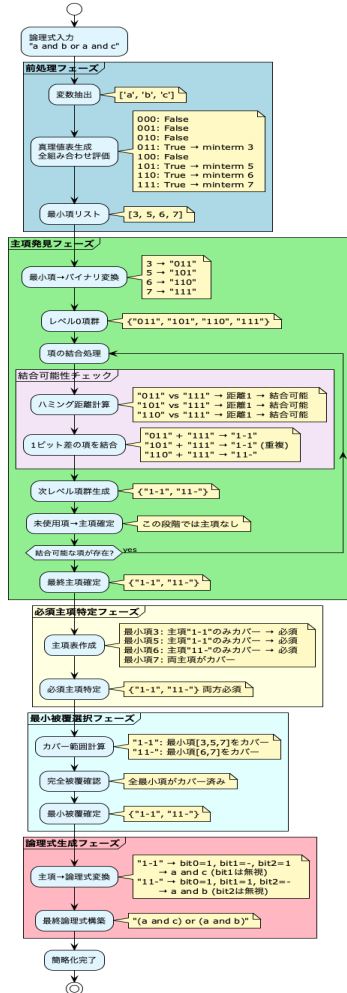
# リファクタリング（パターン2）：ソースコードのリファクタリング



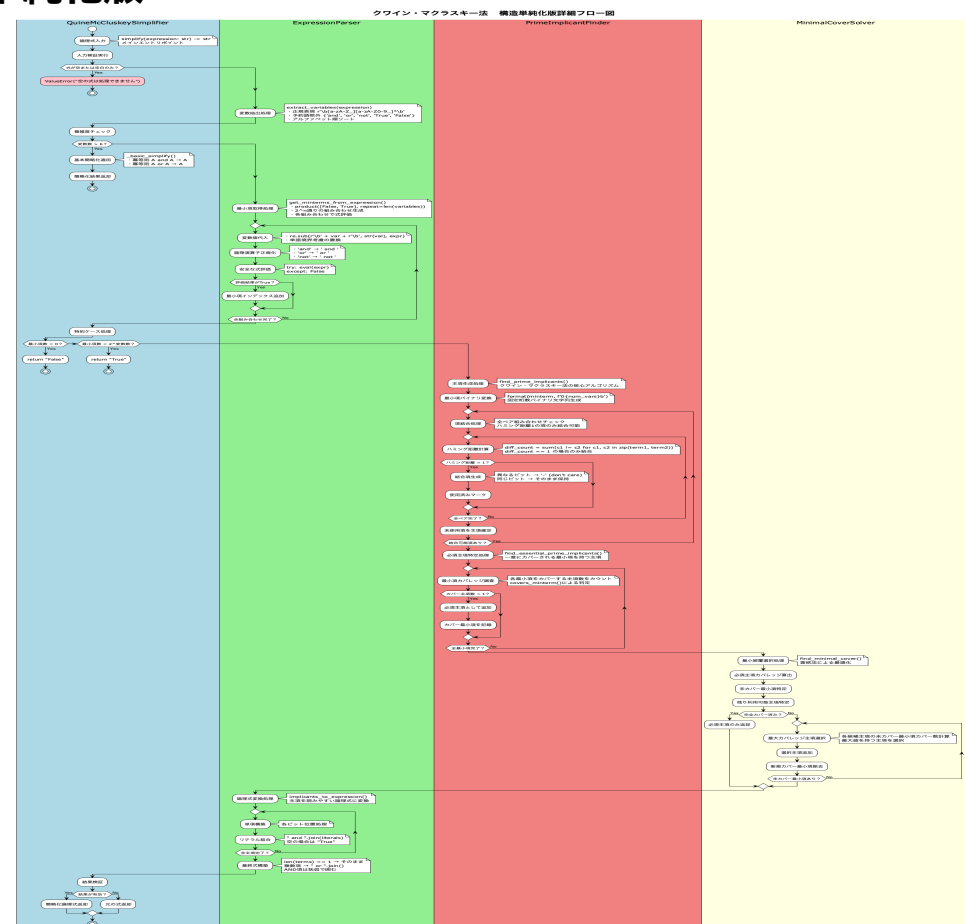
- リファクタリング提案1) のフローチャート（詳細）の比較：コードを分割することでフローが分割された様子。

## オリジナル

クワイン・マクラスキー法 詳細データフロー図



## 構造単純化版





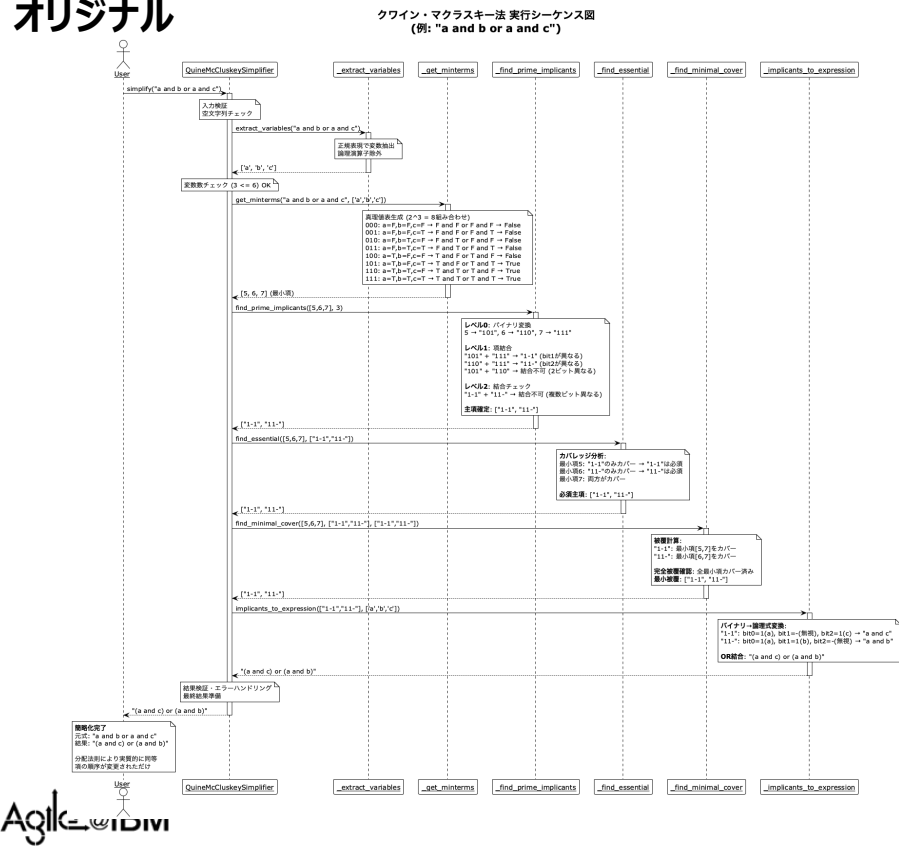
# リファクタリング（パターン2）：ソースコードのリファクタリング



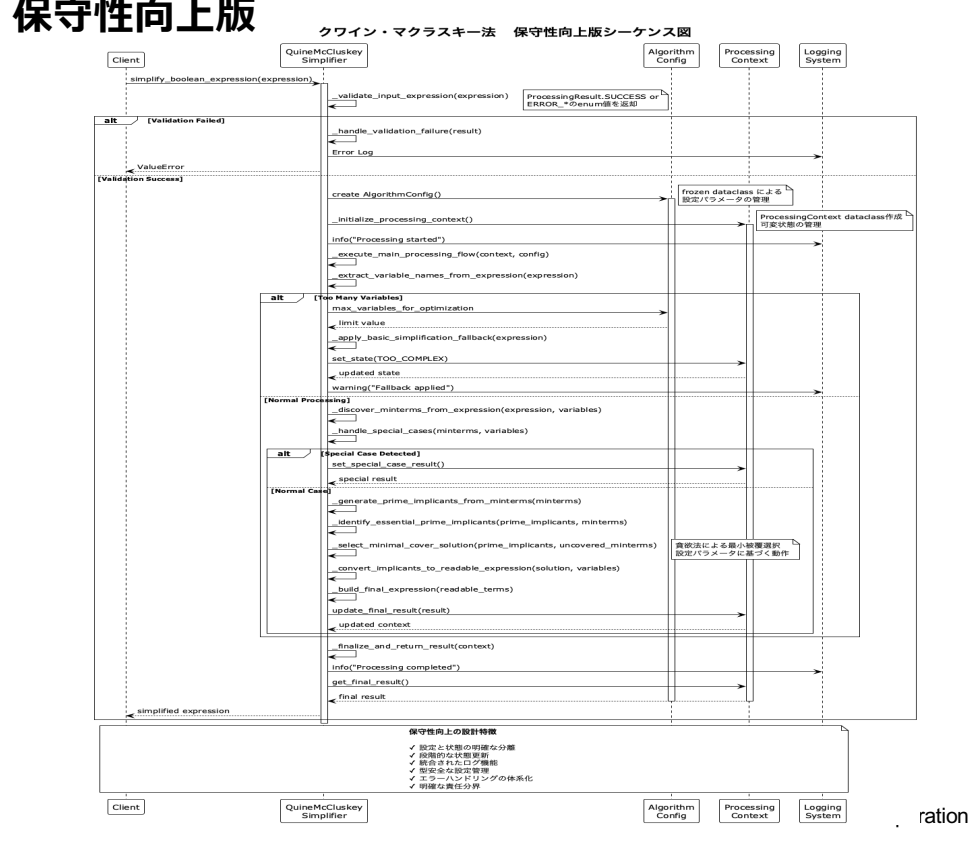
- リファクタリング提案 1) のシーケンス図の比較：保守性向上版のほうが循環的複雑度が増加した傾向と符合します。
- 一方で保守性向上版の方が「認知的複雑度」を低減させている数値が出ているので「保守しやすい」とも言えます。

| 提案名    | LOC | 循環的複雑度 | 認知的複雑度 | メソッド数 | 総合評価  |
|--------|-----|--------|--------|-------|-------|
| オリジナル  | 451 | 79     | 206    | 10    | -     |
| 保守性向上版 | 512 | 97     | 170    | 22    | ★★★★☆ |

## オリジナル



## 保守性向上版



ration

## フォワード生成（パターン2）：コード生成（リファクタリング）



- ・ 生成したコードについて、指定したテストが実施され、それぞれの検証結果で完全一致であることが確認できました。
- ・ さらに、「拡張の余地・改善点」について多くの提案が得られました。

### 提案1: 速度最優先実装（`filter_for_part_of_speech_by_mecab_speed_optimized`）

特徴:

set()を使用して品詞検索をO(1)に最適化  
リスト内包表記による高速フィルタリング  
事前の形式チェックで例外処理を回避

メリット:

大量データ処理に最適  
メモリ効率が良好  
処理速度が最速

デメリット:

可読性がやや劣る  
デバッグが困難

### 提案2: 保守性最優先実装（`filter_for_part_of_speech_by_mecab_maintainable`）

特徴:

MeCabLineParserと  
PartOfSpeechFilterクラスによる責  
任分離  
単一責任原則に基づく設計  
明確なメソッド名と型ヒント

メリット:

チーム開発に最適  
機能拡張が容易  
コードの意図が明確

デメリット:

オーバーヘッドが若干増加  
小規模プロジェクトには過度

### 提案3: テスト性最優先実装（`filter_for_part_of_speech_by_mecab_testable`）

特徴:

各処理ステップを独立した純粋関数に分  
離  
副作用のない設計  
最小12テストケースで完全カバレッジ

メリット:

単体テストが容易  
デバッグが簡単  
品質保証に最適

デメリット:

関数数が増加  
若干のパフォーマンスオーバーヘッド

© 2017 International Business Machines Corporation

## 例外処理生成（パターン2） 4



- 例外処理の実装に関してはチームや開発組織のルールが存在します。また汎用のフレームワークが存在する場合などもあり、絶対的な解はありません。
- ここでは入力チェック処理や結果0件等に対するプリミティブな例外処理行の追加を記載します。

### – 生成AIに例外処理生成を依頼した結果プロンプト

| 考慮すべき例外処理       | 具体的なPythonの例外クラス                                                  | 関連するテクニック/例文                                                                                                                  |
|-----------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| ファイル操作や入出力時のエラー | FileNotFoundError, IOError, PermissionError, IsADirectoryError    | ファイルのオープン、読み書き時に発生し得ます。with open(...)構文を使用することで、例外発生時でもファイルが確実に閉じられます。                                                        |
| 数値演算時のエラー       | ZeroDivisionError, OverflowError                                  | ゼロ除算や、計算結果が型の表現可能な範囲を超えた場合に発生します。特にユーザー入力に基づいた計算で重要です。                                                                        |
| 型変換や値の不一致       | ValueError, TypeError, KeyError, IndexError                       | ユーザー入力を数値に変換する際（ValueError）、関数に誤った型の引数を渡した際（TypeError）、辞書に存在しないキーにアクセスした際（KeyError）、リストの範囲外のインデックスにアクセスした際（IndexError）に利用します。 |
| 反復処理における終了      | StopIteration                                                     | イテレータの次の要素がない場合に、内部的に発生します。forループはこの例外を自動的に処理しますが、next()関数を直接使う場合は処理が必要です。                                                    |
| 外部システム連携時のエラー   | requests.exceptions.RequestException, sqlite3.Error, TimeoutError | Web APIへのリクエスト失敗、データベース接続・操作エラー、ネットワークタイムアウトなど、外部とのやり取りで発生する固有の例外です。外部ライブラリ固有の例外を適切に捕捉します。                                    |
| 実行環境やシステム関連のエラー | MemoryError, RecursionError, SystemExit, KeyboardInterrupt        | メモリ不足、再帰の深さ超過、明示的な終了、Ctrl+Cによる中断など、システムレベルで発生します。これらを捕捉することで、終了前のクリーンアップ処理（リソース解放など）を実行できます。                                  |
| 前提条件を満たさないエラー   | AssertionError                                                    | プログラムのデバッグ時や、内部状態の整合性チェックのために使用します。本番環境では無効にされることがあるため、必須の検証には使いません。                                                          |
| 予期しない一般的なエラー    | Exception                                                         | 広範な例外を捕捉する際に使用しますが、最小限にとどめるべきです。特定の例外処理で対応できない場合に、ログ出力などの処理を行い、プログラムのクラッシュを防ぎます。                                              |

Agile@IBM – ここをプロンプトに記述する

## 例外処理生成（パターン2） 4



- 例外処理の実装に関してはチームや開発組織のルールが存在します。また汎用のフレームワークが存在する場合などもあり、例外処理に実装エラーが含まれる場合も少なくありません。追加要件に「例外実装が迷う場合は」と記載するのがコツ。

添付Pythonコードに対して例外処理を追加してください。

処理の中で発生する可能性がある以下の例外処理については例外処理を追加してください。

実装したい例外処理:

- ファイル操作や入出力時のエラー: FileNotFoundError, IOError, PermissionError, IsADirectoryError
- 数値演算時のエラー: ZeroDivisionError, OverflowError
- 型変換や値の不一致: ValueError, TypeError, KeyError, IndexError
- 反復処理における終了: StopIteration
- 外部システム連携時のエラー: requests.exceptions.RequestException, sqlite3.Error, TimeoutError
- 実行環境やシステム関連のエラー: MemoryError, RecursionError, SystemExit, KeyboardInterrupt
- 前提条件を満たさないエラー: AssertionError
- 予期しない一般的なエラー: Exception

追加要件:

- 結果として例外処理を追加したコードは、同名のファイル名に「\_with\_Exceptions.py」を付与して元のファイルとは別ファイルに保存してください
- 例外処理が一意に決定できない場合は「# TODO: 再確認の必要あり」「未実装」などのコメントと共にpass行を挿入してください

quine\_mcc  
luskey.py  
コメント解説  
付きコード

quine\_mcc  
luskey\_with  
\_Exception  
s.py  
例外付きコード

- 一般的に、例外処理の追記には、**コードに対する例外記述追加の方法と、例外処理クラスの開発をAIに指示し、例外クラスをImport する方法と2種類があります。**
- 大規模プロジェクトであれば、標準的な手法があるかもしれませんが、まずは小さくコードをStepByStepで理解するために、コードへの例外追記の手法をお勧めします。その後汎用的な再利用可能な例外処理をライブラリ化の方が、標準化の観点で望ましいです。

## 例外処理生成（パターン2） 4



- 生成されたコードは例外処理が大量に追加され、メトリクスデータは大きく変化したものの、コードの堅牢性向上、特に「防御的プログラミング：各処理段階での検証」「エラー分類：適切なエラー型での例外発生」などの点で大きく改善されました。

### ## 主要指標分析

#### ### コード規模

- コード行数 (LOC): 451 → 1026 行 (+127.5%)
- 例外処理の追加により、コードサイズが約2.3倍に増加

#### ### 複雑度

- 循環複雑度: 79 → 219 (+177.2%)
- 認知複雑度: 206 → 835 (+305.3%)
- 例外処理の追加により複雑度が大幅に増加

#### ### コメント・ドキュメント

- コメント率: 41.46% → 26.71% (-14.75ポイント)
- コード量増加に対してコメント率は相対的に減少

| Metrics id | Metrics Name                        | File1<br>(Original) | File2<br>(Exception) |
|------------|-------------------------------------|---------------------|----------------------|
| 1          | LOC                                 | 451                 | 1026                 |
| 2          | NCSS                                | 291                 | 729                  |
| 3          | # of Null Lines                     | 86                  | 167                  |
| 4          | Comment Rate                        | 41.46               | 26.71                |
| 5          | Number of Operand                   | 144                 | 310                  |
| 6          | Number of Operator                  | 27                  | 30                   |
| 7          | Number of innerClass                | 0                   | 0                    |
| 8          | Number of Method                    | 10                  | 10                   |
| 9          | Cyclomatic Complexity (Class Total) | 79                  | 219                  |
| 10         | Cognitive Complexity (Class Total)  | 206                 | 835                  |

### ## 品質向上ポイント

#### ### 改善された点

- 堅牢性の向上: 包括的な例外処理により、エラー耐性が大幅に向上
- エラーハンドリング: 10種類以上の例外型に対応した詳細な処理
- ログ機能: 問題発生時の追跡とデバッグが容易
- 型安全性: 引数の型チェックによる実行時エラーの防止
- メモリ管理: メモリ不足やオーバーフローの監視

#### ### 注意点

- コード複雑度の増加: 認知複雑度が3倍以上増加
- 保守コストの増加: コード量の増加により保守負荷が上昇
- パフォーマンス: 多重の例外チェックによる実行速度への影響

### ## 推奨事項

#### ### 最適化の提案

- 複雑度の分散: 一部の例外処理をヘルパーメソッドに分離
- ログレベルの調整: 本番環境では詳細ログを制限
- 選択的例外処理: 重要度に応じた例外処理の段階的適用
- コメント強化: 複雑な例外処理部分の説明を追加

#### ### 運用指針

- 開発環境: 詳細な例外処理版を使用
- 本番環境: パフォーマンス要件に応じて最適化版を検討
- テスト: 例外パスの網羅的なテストケース作成



## 複雑度軽減（パターン2）2

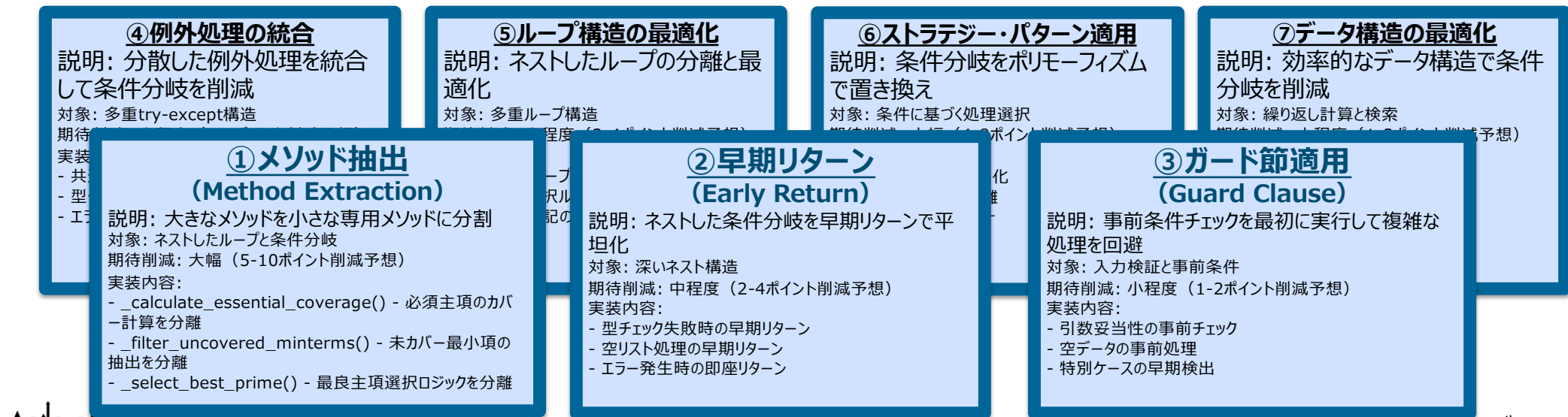


- ソースコードのリファクタリング処理では、複雑度を意識しながらコードの書き直しを行いました。この書き直しの延長として複雑度を下げる書き直し「だけ」を部分的に実施します。複雑度軽減リファクタリングは、テストケース数を劇的に削減できる場合があるため、典型的な等価書き直しの手法として古くから使われています。

例外処理を付け加えた添付ファイルのうち、「\_find\_minimal\_cover」メソッドについて、複雑度軽減のためのリファクタリングを提案してください

- 適用可能な循環的複雑度の低減策にどんなものがあるか列挙してください。
- 元のメソッドの循環的複雑度を計測し（./QI\_Python\_CyclomaticComplexity.pyを利用）、1.で示した各提案を実施したらどの程度循環的複雑度が低減できるかテスト実施してください。

### － 提案された複雑度低減策：7種類



## 複雑度軽減（パターン2）2



- ソースコードのリファクタリング処理では、複雑度を意識しながらコードの書き直しを行いました。この書き直しの延長として複雑度を下げる書き直し「だけ」を部分的に実施します。複雑度軽減リファクタリングは、テストケース数を劇的に削減できる場合があるため、典型的な等価書き直しの手法として古くから使われています。

| refactoring_name | description                                                                                                                                                                                                                         | cyclomatic complexity | Reduction_from original(pt) | reduction percentage(%) |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------------------------|-------------------------|
| 元のメソッド           | 例外処理強化版の元のメソッド                                                                                                                                                                                                                      | 42                    | 0                           | 0.0                     |
| 1_メソッド抽出         | 大きなメソッドを小さな専用メソッドに分割（期待削減：大幅（5-10ポイント削減予想））<br>- <code>_calculate_essential_coverage()</code> - 必須主項のカバー計算を分離<br>- <code>_filter_uncovered_minterms()</code> - 未カバー最小項の抽出を分離<br>- <code>_select_best_prime()</code> - 最良主項選択ロジックを分離 | 11                    | 31                          | 73.8                    |
| 2_早期リターン         | ネストした条件分岐を早期リターンで平坦化（期待削減：中程度（2-4ポイント削減予想））<br>- 型チェック失敗時の早期リターン<br>- 空リスト処理の早期リターン, エラー発生時の即座リターン                                                                                                                                  | 31                    | 11                          | 26.2                    |
| 3_ガード節           | 事前条件チェックを最初に実行（期待削減：小程度（1-2ポイント削減予想））<br>- 引数妥当性の事前チェック<br>- 空データの事前処理, 特別ケースの早期検出                                                                                                                                                  | 27                    | 15                          | 35.7                    |
| 4_例外処理統合         | 分散した例外処理を統合して条件分岐を削減（期待削減：中程度（3-5ポイント削減予想））<br>- 共通例外処理の統合<br>- 型チェック例外の一元化 + エラーハンドリングの簡素化                                                                                                                                         | 29                    | 13                          | 31.0                    |
| 5_ループ最適化         | ネストしたループの分離と最適化（期待削減：中程度（2-4ポイント削減予想））<br>- カバー計算ループの分離<br>- 最良主項選択ループの独立化<br>- リスト内包表記の活用                                                                                                                                          | 18                    | 24                          | 57.1                    |
| 6_戦略パターン         | 条件分岐をポリモーフイズムで置き換え（期待削減：大幅（4-8ポイント削減予想））<br>- 主項選択戦略の抽象化<br>- カバー計算戦略の分離<br>- エラー処理戦略の統一                                                                                                                                            | 15                    | 27                          | 64.3                    |
| 7_データ構造最適化       | 効率的なデータ構造で条件分岐を削減（期待削減：小程度（1-3ポイント削減予想））<br>- <code>set</code> 使用による検索最適化 + <code>dict</code> によるマッピング活用<br>- キャッシュ機構の導入                                                                                                            | 21                    | 21                          | 50.0                    |